

Actividad 4 — Seguridad en MongoDB: autenticación, roles y NoSQL injection

Datos generales

- **Duración:** 50 minutos
- **Tipo:** Individual
- **Herramienta de IA:** Libre
- **Requisitos:** MongoDB en Docker (Actividad 2)

Presupuesto de tiempo

Paso	Tiempo
Leer el marco teórico	7 min
Crear usuario administrador	8 min
Habilitar autenticación en el contenedor	10 min
Crear usuarios con roles	8 min
Simular NoSQL injection	12 min
Verificación y entrega	5 min

Objetivos

1. Entender los pilares de seguridad en una base NoSQL: **autenticación, autorización, cifrado y validación de entradas**.
2. Habilitar autenticación en MongoDB y crear usuarios con **roles** (RBAC).
3. Comprender qué es una **NoSQL injection** y cómo prevenirla.
4. Aplicar el principio de **mínimo privilegio**.

Marco teórico

Seguridad en bases NoSQL

Una base de datos expuesta en la red sin protección es un riesgo serio: cualquier persona podría conectarse, leer, borrar o cifrar los datos (ransomware). Los controles se dividen en capas:

Capa	Control	Respuesta en MongoDB
Autenticación	¿Quién es?	Usuarios y contraseñas (scram-sha-256)
Autorización	¿Qué puede hacer?	Roles RBAC (readWrite, read, root...)
Red	¿Desde dónde?	bindIp, firewall, puertos

Capa	Control	Respuesta en MongoDB
Cifrado en tránsito	¿Se lee en el camino?	TLS/SSL
Cifrado en reposo	¿Se lee del disco?	Cifrado en reposo (enterprise) o cifrado de archivos
Validación	¿Los datos son seguros?	Validadores de esquema + sanitización de entradas

RBAC (Role-Based Access Control)

MongoDB asigna **roles** a cada usuario. Los roles agrupan privilegios. Ejemplos:

- read → solo consultas sobre una base.
- readWrite → consultas + escrituras sobre una base.
- userAdminAnyDatabase → administrar usuarios.
- root → todos los privilegios (usar solo para administrar).

Regla de oro: el usuario de una aplicación nunca debe usar root. Cada aplicación usa su propio usuario con readWrite solo sobre su base.

¿Qué es una NoSQL injection?

Muy parecida a la SQL injection, pero ataca el **operador de consulta** de MongoDB. Si una aplicación arma consultas concatenando texto sin validar, un atacante puede inyectar operadores como \$ne, \$gt o \$where y lograr:

- Acceso sin conocer la contraseña.
- Exfiltración de todos los registros.
- Denegación de servicio con \$where pesado.

Prevención: nunca concatenar entradas del usuario en filtros. Usar parámetros / construcciones con operadores controlados y validar tipos (la entrada del usuario es **dato**, no consulta).

Paso a paso

Paso 1 — Levantar MongoDB con autenticación

Borrá el contenedor anterior y crealo **habilitando la autenticación**:

```
docker rm -f mongo-clase
```

```
docker run -d --name mongo-clase \
  -p 27017:27017 \
  -v mongo-clase-data:/data/db \
  mongo:7 --auth
```

- --auth hace que MongoDB exija autenticación para conectarse.

Paso 2 — Crear el usuario administrador

MongoDB con --auth **no tiene ningún usuario inicial**. Entramos una vez "desprotegidos" pero **solo contra localhost** (el contenedor solo se puede tocar desde tu PC):

```
docker exec -it mongo-clase mongosh
```

```
use admin
db.createUser({
  user: "admin",
  pwd: "Admin-Muy-Secreto-2026",
  roles: [ { role: "root", db: "admin" } ]
})
exit
```

Paso 3 — Conectar con credenciales

Ahora la conexión pide usuario y contraseña:

```
docker exec -it mongo-clase mongosh "mongodb://admin:Admin-Muy-Secreto-2026@localhost:27017/admin"
```

En una terminal normal (fuera del contenedor) también podés conectarte porque el puerto está publicado: `mongosh "mongodb://admin:Admin-Muy-Secreto-2026@127.0.0.1:27017/admin"` (si tenés mongosh instalado en el PC).

Paso 4 — Crear usuarios de aplicación con roles

Mientras estás conectado como admin, creá una base y un usuario **con mínimo privilegio**:

```
// Base de la aplicación
```

```
use app_ventas
```

```
// Usuario de solo lectura
```

```
db.createUser({
  user: "lector",
  pwd: "Lector123",
  roles: [ { role: "read", db: "app_ventas" } ]
})
```

```
// Usuario de lectura y escritura (el que usa la aplicación)
```

```
db.createUser({
  user: "app_ventas_user",
  pwd: "AppVentas-Secreto",
```

```
    roles: [ { role: "readWrite", db: "app_ventas" } ]
  })
```

Paso 5 — Probar los roles

Conectate como app_ventas_user y probá leer y escribir:

```
docker exec -it mongo-clase mongosh "mongodb://app_ventas_user:AppVentas-Secreto@localhost:27017/app_ventas"
```

```
db.productos.insertOne({ nombre: "Laptop", precio: 900 })
db.productos.find().pretty()
```

Ahora probá conectarte como lector e intentá **escribir** (debe dar error not authorized):

```
docker exec -it mongo-clase mongosh "mongodb://lector:Lector123@localhost:27017/app_ventas"
```

```
db.productos.insertOne({ nombre: "Hack", precio: 1 })
```

Deberías ver un error de autorización. Eso es el RBAC funcionando.

Paso 6 — Ver qué roles tiene cada usuario

```
docker exec -it mongo-clase mongosh "mongodb://admin:Admin-Muy-Secreto-2026@localhost:27017/admin"
```

```
use admin
db.getUsers()
```

Paso 7 — Simular una NoSQL injection

Creá una colección de usuarios de ejemplo con credenciales:

```
use app_ventas
db.usuarios.insertMany([
  { usuario: "ana", clave: "s3creto", admin: false },
  { usuario: "bruno", clave: "otra", admin: true }
])
```

Imaginá una página de login que arma el filtro así (nunca hagas esto):

```
// CÓDIGO VULNERABLE (para entender el ataque):
// const filtro = { usuario: user, clave: pass };
// db.usuarios.findOne(filtro)
```

Si el atacante ingresa user = { "\$ne": null } y pass = { "\$ne": null }, el filtro queda así y **devuelve el primer usuario sin saber ninguna contraseña**:

```
// Simulación del ataque: esto es lo que la app vulnerable ejecutaría
db.usuarios.findOne({ usuario: { $ne: null }, clave: { $ne: null } })
```

Ese comando devuelve un documento aunque el atacante no conozca credenciales.

Paso 8 — Ver el equivalente SQL

La misma idea en SQL:

```
-- SELECT * FROM usuarios WHERE usuario = '' OR '1'='1' AND clave = ''  
-- El '1'='1' hace que el WHERE sea siempre verdadero
```

Paso 9 — Prevenir la inyección

Las tres defensas esenciales:

1. **Nunca** concatenar texto del usuario en filtros: usar operadores fijos y comparar valores por igualdad exacta.
2. **Validar tipos** antes de usarlos (si se espera un string, rechazar objetos).
3. **Validación de esquema** con JSON Schema para que la base rechace documentos con campos raros.

Ejemplo de validador para que db.usuarios solo acepte strings en usuario y clave:

```
db.runCommand({  
  collMod: "usuarios",  
  validator: {  
    $jsonSchema: {  
      bsonType: "object",  
      required: ["usuario", "clave"],  
      properties: {  
        usuario: { bsonType: "string" },  
        clave: { bsonType: "string" }  
      }  
    }  
  }  
})
```

Probá que ahora rechaza un objeto en usuario:

```
db.usuarios.insertOne({ usuario: { $ne: null }, clave: "x" })  
// → debe fallar con la validación de esquema
```

Paso 10 — Preguntar a la IA

¿Qué es el principio de mínimo privilegio y cómo se aplica con los roles de MongoDB?

¿Cuál es la diferencia entre cifrado en tránsito y cifrado en reposo?

Mostrame un ejemplo de cómo una API debería validar un login sin exponerse a NoSQL injection.

Verificación de resultados

- ☐ Con --auth, una conexión sin credenciales no puede hacer nada.

- ☐ El usuario lector no puede insertar (error not authorized).
- ☐ El usuario app_ventas_user puede leer y escribir.
- ☐ La simulación de inyección devuelve un usuario sin conocer la clave.
- ☐ El validador de esquema rechaza documentos con operadores.

Criterios de evaluación

Criterio	Puntos
Contenedor con --auth levantado	15
Usuario admin + usuarios de aplicación creados	25
Prueba de RBAC (lector no puede escribir)	20
Simulación y explicación de NoSQL injection	20
Validador de esquema + preguntas a la IA	20

Entregable

- Archivo seguridad-mongodb.md con: usuarios creados, capturas de los errores de autorización, la simulación de inyección y el validador.
- Reflexión de 5 líneas sobre por qué una API no debe armar filtros con texto del usuario.

Para pensar

En la Actividad 6 vas a ver la **seguridad en Redis** (que históricamente es aún más abusada, porque se lo expone sin contraseña y con comandos peligrosos). La Actividad 12 unificará todo con docker compose y secretos.